BIG DATA ANALYTICS LAB

LAB MANUAL

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Course Code & Name	: 102008/AD701S – BIG DATA ANALYTICS LAB
Regulations	: 2023 – Autonomous
Semester	: VII
Program	: ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
Prepared by	: Dr. Jithin Mathews

RAJAGIRI SCHOOL OF ENGINEERING AND TECHNOLOGY(AUTONOMOUS)

VISION

To evolve into a premier technological and research institution, molding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

MISSION

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATASCIENCE

VISION

To become a center of excellence in Artificial Intelligence and Data Science and moulding professionals by imparting high-quality knowledge for the sustainable betterment of mankind.

MISSION

To empower individuals with expertise in constantly evolving AI & Data Science tools and technologies for impactful innovation, research and societal advancement through ethical practices and industrial collaborations.

Program Outcomes

- **PO1. Engineering Knowledge:** Apply mathematics, science, and engineering fundamentals to complex problems.
- **PO2. Problem Analysis:** Analyze complex problems and reach substantiated conclusions.
- **PO3. Design/Development of Solutions:** Create solutions that meet specific public health, safety, and environmental needs.
- **PO4. Conduct Investigations of Complex Problems:** Use research-based methods, including experiments and data analysis.
- **PO5. Engineering Tool Usage:** Utilize modern engineering and IT tools.
- **PO6. The Engineer and the World:** Understand the impact of engineering solutions in societal, environmental, and global contexts.
- **PO7. Ethics:** Apply ethical principles and understand human values, diversity, and inclusion.
- **PO8. Individual and Collaborative Team Work:** Function effectively as an individual or team leader in diverse environments.
- **PO9. Communication:** Communicate effectively regarding complex engineering activities.
- **PO10. Project Management and Finance:** Apply management and financial principles to engineering projects.
- **PO11. Life-Long Learning:** Engage in independent and continuous learning to adapt to technological change

Program Specific Objectives Artificial Intelligence and Data Science

PSO1: Apply the fundamentals of science, engineering and mathematics to understand, analyze and develop solutions in the areas related to artificial intelligence and data science for optimal design of intelligent systems.

PSO2: Design and Implement appropriate techniques and analytic tools for the integration of intelligent systems, with a view to engaging in lifelong learning for the betterment of society.

PSO3: Practice professional ethics in applying scientific method to model and support multi-disciplinary facets of engineering and its societal implications.

COURSE OUTCOMES (CO)

After the completion of the course the student will be able to:

CO1: Demonstrate understanding of big data concepts, ecosystem, and technologies like Hadoop and Spark.

CO2: Install, configure, and operate big data tools such as HDFS, Hive, Pig, and Spark. Develop programs using HADOOP framework.

CO3: Implement MapReduce and Spark programs to process large-scale datasets.

CO4: Analyze the given dataset and identify deep insights from the dataset.

CO5: Develop and Implement innovative ideas on big data technologies.

INSTRUCTIONS TO STUDENTS

1. Students should report to the concerned labs 5 min prior to the given timetable.
2. All the students are supposed to enter the lab with the record.
3. All the students should sit according to their allotted seats given by the respective lab in charge.
4. Do not change the terminal on which you are working.
5. All the students are expected to get at least the algorithm of the program/concept to be implemented.
6. Final experiment signature should be given by the corresponding allotted faculty and they will enter the marks in the sheet given.
7. No student will be allowed to write the model exam without completing the fair record.
8. The fair record must be certified by the coordinator/lab-in-charge before the model exam at the last page of experiments, index page, and certificate page.
9. Strictly observe the instructions given by the teacher/Lab Instructor.
10. When the experiment is completed, students should shut down the computers before leaving the lab.
11. Any damage to the lab computers will be viewed seriously.
12. Students should not leave the lab without the concerned faculty's permission.
13. Do not disturb machine Hardware / Software Setup.

RSET
RAJAGIRI SCHOOL OF ENGINEERING & TECHNOLOGY (AUTONOMOUS)

STUDENT MANUAL

Note:

- Students need to write the **Title, Aim, Theory, Algorithm**, and **Conclusion** in the Lab record). The **Script/Code and Output** must be printed and pasted on the left side of the record.
- Title, Aim and Theory are given in the manual below. The remaining 4 sections must be written by the students based on the code that they are writing.

LAB#1

Title: "Experiment 1: Downloading and Installing Hadoop: Modes, Startup Scripts, and Configuration Files"

Aim:

To download and install Apache Hadoop on Ubuntu, understand its different operating modes, and learn the purpose of its startup scripts and configuration files by completing the following tasks:

1. Install the prerequisites (Java and SSH) and set up password-less SSH login to localhost.
2. Download and install Apache Hadoop, and configure its environment variables.
3. Run Hadoop in Standalone (Local) mode to verify the installation.
4. Edit Hadoop's configuration files to set it up in Pseudo-distributed mode.
5. Format the HDFS filesystem, start the daemons using the startup scripts, and verify that the cluster is running.

Theory:

What is Hadoop?

Apache Hadoop is an open-source framework that allows the distributed storage and distributed processing of very large datasets across a cluster of ordinary computers. Instead of depending on one powerful (and expensive) machine, Hadoop spreads both the data and the computation across many cheap machines, and is designed to keep working even when some of those machines fail.

Hadoop has three core parts that matter for this lab:

- HDFS (Hadoop Distributed File System) - the storage layer. It splits a large file into blocks and stores them across the cluster.
- YARN (Yet Another Resource Negotiator) - the resource manager. It decides which machine runs which piece of work.
- MapReduce - the original processing model that runs jobs on top of YARN.

Why we install the prerequisites (Task 1)?

Hadoop is written in the Java programming language, so a Java Development Kit (JDK) must be present before Hadoop can run - this is why we install OpenJDK. Hadoop also starts and stops its background programs (called daemons) by logging into machines over SSH. Even on a single computer, the startup scripts log in to "localhost" (the same machine) using SSH. If SSH asks for a password every time, the scripts will stall. We therefore install an SSH server and create a password-less key so the scripts can log in automatically.

Why we download and set environment variables (Task 2)?

Hadoop is distributed as a ready-to-use compressed binary package (a .tar.gz tarball). "Installing" it simply means extracting this package into a folder. We then set environment variables so the operating system knows where Java is (JAVA_HOME), where Hadoop is (HADOOP_HOME), and so that Hadoop's commands can be typed from any folder (PATH).

Understanding the three Hadoop modes (Tasks 3 and 4):

A key goal of this experiment is to understand the three modes in which Hadoop can run:

Standalone (Local) mode: This is the default right after extraction. No daemons run, HDFS is not used, and everything executes inside a single Java process using ordinary local files. It is the simplest mode and is used to quickly test that the installation works and to debug program logic.

Pseudo-distributed mode: Each Hadoop daemon runs as a separate Java process, but all of them run on one single machine. It uses HDFS and behaves like a real cluster squeezed onto one computer. This is the standard mode for learning, and the one we configure in this lab.

Fully distributed mode: The daemons run across many physical machines. This is what is used in real production clusters. We only discuss it here; we do not build it in the lab.

Understanding the configuration files (Task 4):

Hadoop's behaviour is controlled by a few text files inside the etc/hadoop folder. The important ones are:

hadoop-env.sh - sets environment values (most importantly JAVA_HOME) for the Hadoop daemons themselves.

core-site.xml - core settings. We set fs.defaultFS, which tells Hadoop that the default filesystem is HDFS running at localhost on port 9000.

hdfs-site.xml - HDFS settings. We set dfs.replication to 1, meaning each block of data is stored as only one copy (we have just one machine, so more copies are pointless).

mapred-site.xml - MapReduce settings. We tell it to run jobs using YARN.

yarn-site.xml - YARN settings, including the shuffle service that MapReduce needs.

Startup scripts and verification (Task 5):

Before HDFS can be used the very first time, its metadata area must be initialised - this is done by formatting the NameNode. Hadoop then provides shell scripts to start and stop the daemons:

start-dfs.sh starts the storage daemons: the NameNode (the master that tracks where data lives), the DataNode (stores the actual data), and the SecondaryNameNode (a housekeeping helper).

start-yarn.sh starts the processing daemons: the ResourceManager (master) and the NodeManager (worker).

stop-dfs.sh and stop-yarn.sh stop them again.

The jps command lists all running Java processes and is used to confirm that every daemon started correctly.

Algorithm:

The high-level steps to complete this experiment are:

1. Update the system's package list, then install OpenJDK 11 and the OpenSSH server.
2. Generate an SSH key pair, authorise it for localhost, and confirm password-less login works.
3. Download the Hadoop binary tarball, extract it, and move it to /usr/local/hadoop.
4. Define the environment variables (JAVA_HOME, HADOOP_HOME, PATH) and set JAVA_HOME inside hadoop-env.sh; verify with hadoop version.
5. Run a built-in example job in Standalone mode to confirm the installation works.
6. Edit core-site.xml, hdfs-site.xml, mapred-site.xml and yarn-site.xml to switch to Pseudo-distributed mode.
7. Format the HDFS NameNode (first time only).
8. Start the HDFS and YARN daemons with the startup scripts and verify with jps and the web dashboards.
9. Stop all daemons cleanly when finished.

Code:

Run the commands below in the Ubuntu terminal in the given order. Each block is explained in simple language. Lines beginning with # are comments and do not need to be typed.

Task 1 - Install prerequisites and set up password-less SSH

Step 1.1 - Install Java and SSH

This refreshes the list of available software, then installs the Java development kit and the SSH server. The -y answers "yes" automatically. After it finishes we check that Java installed correctly.

```
# Refresh the list of downloadable software packages
sudo apt update

# Install Java 11 (needed to run Hadoop) and the SSH server
sudo apt install -y openjdk-11-jdk openssh-server

# Confirm Java is installed (should print a version number)
java -version
```

Step 1.2 - Create a password-less SSH key for localhost

Hadoop's scripts log in to the machine over SSH. Here we create a key with no passphrase, add it to the list of authorised keys, and tighten the file permissions. The final 'ssh localhost' should connect WITHOUT asking for a password.

```
# Generate an SSH key with an empty password (-P '')
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa

# Authorise this key so we can log in to our own machine
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

```
# Set the correct (private) permissions on the authorised keys file
chmod 0600 ~/.ssh/authorized_keys

# Test it - this must connect without asking for a password
ssh localhost

# Return to the original session
exit
```

Task 2 - Download and install Hadoop, and set environment variables

Step 2.1 - Download, extract and place Hadoop

We download the Hadoop package, unpack it, and move it to /usr/local/hadoop, which will be its permanent home.

```
# Go to the home folder
cd ~

# Download the Hadoop 3.4.3 package from the Apache archive
wget https://archive.apache.org/dist/hadoop/common/hadoop-3.4.3/hadoop-3.4.3.tar.gz

# Unpack the downloaded .tar.gz file
tar -xzf hadoop-3.4.3.tar.gz

# Move the extracted folder to /usr/local and rename it to 'hadoop'
sudo mv hadoop-3.4.3 /usr/local/hadoop
```

Step 2.2 - Set the environment variables

These lines tell the system where Java and Hadoop live, and add Hadoop's commands to the PATH so they can be run from anywhere. We append them to ~/.bashrc (a file that runs each time a terminal opens), then reload it.

```
# Append Hadoop's environment variables to the end of ~/.bashrc
cat >> ~/.bashrc << 'EOF'

# ----- Hadoop environment variables -----
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
EOF

# Reload .bashrc so the new variables take effect now
source ~/.bashrc
```

Step 2.3 - Tell Hadoop where Java is, and verify

Hadoop does not always inherit JAVA_HOME over SSH, so we also set it inside Hadoop's own environment file. Then 'hadoop version' confirms the install.

```
# Add JAVA_HOME inside Hadoop's own environment file
echo 'export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64' >>
$HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

```
# Verify the installation (should print 'Hadoop 3.4.3')
hadoop version
```

Task 3 - Run Hadoop in Standalone mode

Standalone mode needs no configuration and no running daemons. Here we copy Hadoop's own XML files into an input folder and run a built-in example that searches the text for words matching a pattern (any word starting with 'dfs'). The result is written to an output folder.

```
# Create an input folder and put some sample text files in it
mkdir ~/input
cp $HADOOP_HOME/etc/hadoop/*.xml ~/input

# Run the built-in 'grep' example (this is a real MapReduce job)
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar \
  grep ~/input ~/output 'dfs[a-z.]+'

# Show the result
cat ~/output/*
```

Note: If you run this again, first delete the output folder with rm -r ~/output because Hadoop refuses to overwrite an existing output folder.

Task 4 - Configure Pseudo-distributed mode

Now we switch Hadoop to pseudo-distributed mode by writing four configuration files. Each command below replaces the file's contents with the correct settings. The quoted 'EOF' keeps the text exactly as written.

Step 4.1 - core-site.xml (default filesystem)

```
cat > $HADOOP_HOME/etc/hadoop/core-site.xml << 'EOF'
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
EOF
```

Step 4.2 - hdfs-site.xml (one copy of each block)

```
cat > $HADOOP_HOME/etc/hadoop/hdfs-site.xml << 'EOF'
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
EOF
```

Step 4.3 - mapred-site.xml (use YARN)

```
cat > $HADOOP_HOME/etc/hadoop/mapred-site.xml << 'EOF'
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <property>
    <name>mapreduce.application.classpath</name>
    <value>$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/*:$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/lib/*</value>
  </property>
</configuration>
EOF
```

Step 4.4 - yarn-site.xml (shuffle service)

```
cat > $HADOOP_HOME/etc/hadoop/yarn-site.xml << 'EOF'
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.env-whitelist</name>
    <value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME,HADOOP_MAPRED_HOME</value>
  </property>
</configuration>
EOF
```

Task 5 - Format HDFS, start the daemons, and verify

Step 5.1 - Format the NameNode (first time only)

This initialises the empty HDFS filesystem. Do this only once during the first setup.

```
# Initialise the HDFS filesystem metadata
hdfs namenode -format
```

Step 5.2 - Start the daemons using the startup scripts

These two scripts start the storage and processing daemons. The jps command then lists the running Java processes so we can confirm everything started.

```
# Start HDFS daemons (NameNode, DataNode, SecondaryNameNode)
start-dfs.sh

# Start YARN daemons (ResourceManager, NodeManager)
start-yarn.sh

# List running Java processes to verify all daemons are up
jps
```

After this, open a browser and visit http://localhost:9870 for the HDFS (NameNode) dashboard and http://localhost:8088 for the YARN (ResourceManager) dashboard.

Step 5.3 - Stop the daemons cleanly

When you are finished, stop the daemons in the reverse order.

```
# Stop the YARN daemons
stop-yarn.sh

# Stop the HDFS daemons
stop-dfs.sh
```

Output:

The following are sample outputs (representative terminal screenshots) showing what appears when each step is run successfully. Exact version numbers and process IDs will vary from machine to machine.

Output of Step 1.1 - java -version

```
student@ubuntu: ~

$ java -version
openjdk version "11.0.24" 2024-07-16
OpenJDK Runtime Environment (build 11.0.24+8-post-Ubuntu-1ubuntu1)
OpenJDK 64-Bit Server VM (build 11.0.24+8-post-Ubuntu-1ubuntu1, mixed mode)
```

Output of Step 1.2 - ssh localhost (no password asked)

```
student@ubuntu: ~

$ ssh localhost
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 6.5.0-generic x86_64)

 * Documentation:  https://help.ubuntu.com
Last login: Thu Jun 25 10:14:02 2026 from 127.0.0.1
$ exit
logout
Connection to localhost closed.
```

Output of Step 2.3 - hadoop version

```
student@ubuntu: ~

$ hadoop version
Hadoop 3.4.3
Source code repository https://github.com/apache/hadoop.git
Compiled by root on 2026-02-24T10:42Z
Compiled with protoc 3.23.4
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-
3.4.3.jar
```

Output of Task 3 - Standalone grep example

```
student@ubuntu: ~

$ hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-
*.jar \
    grep ~/input ~/output 'dfs[a-z.]+'
...
INFO mapreduce.Job: Running job: job_local1234567890_0001
INFO mapreduce.Job:  map 100% reduce 100%
INFO mapreduce.Job: Job job_local1234567890_0001 completed successfully
...
$ cat ~/output/*
1   dfsadmin
1   dfs.replication
```

Output of Step 5.1 - hdfs namenode -format

```
student@ubuntu: ~

$ hdfs namenode -format
...
INFO common.Storage: Storage directory /tmp/hadoop-student/dfs/name has been
  successfully formatted.
INFO namenode.NameNode: SHUTDOWN_MSG: Shutting down NameNode at
ubuntu/127.0.1.1
```

Output of Step 5.2 - start-dfs.sh, start-yarn.sh and jps

```
student@ubuntu: ~

$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [ubuntu]

$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers

$ jps
4521 NameNode
4892 SecondaryNameNode
4655 DataNode
5320 ResourceManager
5470 NodeManager
5612 Jps
```

Seeing all five daemons (NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager) listed by jps confirms that Hadoop is correctly installed and running in pseudo-distributed mode.

LAB#2

Title: "Experiment 2.1: HDFS File Management Basics: Adding, Retrieving and Deleting Files and Directories"

Aim:

To perform basic file management operations on the Hadoop Distributed File System (HDFS) using the hdfs dfs command line, by completing the following tasks:

1. Start the Hadoop daemons and prepare a sample file on the local machine.
2. Add files and directories to HDFS (create directories and upload the file from the local machine).
3. Retrieve files from HDFS (list the contents, view a file, and download a file back to the local machine).
4. Delete files and directories from HDFS.

Theory:

What is HDFS?

The Hadoop Distributed File System (HDFS) is the storage layer of Hadoop. It is a special filesystem that stores very large files by splitting them into blocks and spreading those blocks across the machines in a cluster. To the user, however, HDFS looks like an ordinary filesystem with folders (directories) and files arranged in a tree, starting from the root /.

HDFS is separate from the normal Ubuntu (local) filesystem. A file sitting in your home folder is NOT automatically in HDFS - it must be copied in. This is why the first thing we learn is how to move files between the local filesystem and HDFS, and how to manage them once they are there.

The hdfs dfs command

All file operations on HDFS are done through one command family that begins with hdfs dfs, followed by an option that looks like a familiar Linux command. For example:

`hdfs dfs -mkdir` makes a new directory in HDFS.

`hdfs dfs -put` (or -copyFromLocal) uploads a file from the local machine into HDFS.

`hdfs dfs -ls` lists the contents of an HDFS directory.

`hdfs dfs -cat` prints the contents of an HDFS file to the screen.

`hdfs dfs -get` (or -copyToLocal) downloads a file from HDFS to the local machine.

`hdfs dfs -rm` deletes a file, and -rm -r deletes a directory and everything inside it.

Why we perform these tasks

Adding files and directories: Every big data job begins with data. Before Hadoop can process anything, the data must first be placed inside HDFS. Creating directories keeps that data organised, just like folders on a normal computer.

Retrieving files: After data is stored - or after a job produces a result - we need to look at it. Listing shows what exists, viewing shows the contents, and downloading brings a copy back to the local machine so it can be used by other programs.

Deleting files and directories: HDFS has limited space and gets cluttered with old inputs and outputs. Deleting unwanted files and directories is basic housekeeping and is also needed because Hadoop jobs refuse to write to an output folder that already exists.

Understanding HDFS paths

HDFS uses its own paths that are separate from local paths. In this experiment we work under /user/student, which is the conventional home area for a user named 'student' in HDFS. A local path like ~/sample.txt refers to a file on Ubuntu; an HDFS path like /user/student/bdalab/sample.txt refers to a file inside HDFS.

Algorithm:

The high-level steps to complete this experiment are:

1. Start the HDFS and YARN daemons and confirm they are running with jps.
2. Create a small sample text file on the local (Ubuntu) filesystem.
3. Create a home directory and a working directory inside HDFS using -mkdir.
4. Upload the sample file from the local machine into the HDFS directory using -put, and confirm with -ls.
5. List the HDFS directory, view the uploaded file with -cat, and download it back to the local machine with -get.
6. Delete the file with -rm, then delete the whole directory with -rm -r, and confirm the deletion with -ls.

Code:

Run the commands below in the Ubuntu terminal in order. Lines starting with # are comments (explanations) and do not need to be typed. In these commands the user is assumed to be named 'student' - replace it with your own username if different.

Task 1 - Start Hadoop and prepare a sample file

First we make sure the Hadoop daemons from Experiment 1 are running (HDFS cannot be used otherwise). Then we create a small text file on the local machine to use as our sample data.

```
# Start the HDFS and YARN daemons (safe to run even if already started)
start-dfs.sh
start-yarn.sh

# Confirm the daemons are running (look for NameNode and DataNode)
jps

# Create a small sample text file on the LOCAL machine
echo 'Hello Hadoop' >  ~/sample.txt
echo 'This is a HDFS test file.' >> ~/sample.txt
echo 'Big Data Analytics Lab' >> ~/sample.txt

# Show the local file we just created
cat ~/sample.txt
```

Task 2 - Adding files and directories to HDFS

We first create a home directory for the user in HDFS, then a working sub-directory. The -p option creates parent directories as needed. We then upload our local file into HDFS with -put and confirm it arrived with -ls.

```
# Create the user's home directory and a working directory in HDFS
# (-p creates all the parent folders in one go)
hdfs dfs -mkdir -p /user/student/bdalab/data

# Upload the local file 'sample.txt' INTO the HDFS directory
hdfs dfs -put ~/sample.txt /user/student/bdalab/data
```

```
# List the HDFS directory to confirm the file was added
hdfs dfs -ls /user/student/bdalab/data
```

Note: The command `hdfs dfs -copyFromLocal ~/sample.txt /user/student/bdalab/data` does exactly the same thing as -put.

Task 3 - Retrieving files from HDFS

Retrieving means inspecting and downloading data that is stored in HDFS. We list the directory, print the file's contents to the screen with -cat, then download a copy back to the local machine with -get and verify it.

```
# 1) List the contents of the HDFS working directory
hdfs dfs -ls /user/student/bdalab/data

# 2) View the contents of the HDFS file on screen
hdfs dfs -cat /user/student/bdalab/data/sample.txt

# 3) Download (retrieve) the file from HDFS back to the LOCAL machine
hdfs dfs -get /user/student/bdalab/data/sample.txt ~/retrieved.txt

# 4) Confirm the downloaded local copy
cat ~/retrieved.txt
```

Note: The command `hdfs dfs -copyToLocal ...` does the same thing as -get.

Task 4 - Deleting files and directories from HDFS

Finally we clean up. We first delete a single file with -rm, then delete an entire directory (and everything inside it) with -rm -r. A final -ls confirms the directory is gone.

```
# Delete a single file from HDFS
hdfs dfs -rm /user/student/bdalab/data/sample.txt

# Delete an entire directory and all its contents (-r = recursive)
hdfs dfs -rm -r /user/student/bdalab

# Confirm the directory has been deleted (it should no longer be listed)
hdfs dfs -ls /user/student
```

Output:

The following are sample outputs (representative terminal screenshots) showing what appears when each task is run successfully.

Output of Task 1 - daemons running and sample file created

```
student@ubuntu: ~

$ jps
4521 NameNode
4655 DataNode
4892 SecondaryNameNode
5320 ResourceManager
5470 NodeManager
5612 Jps

$ cat ~/sample.txt
Hello Hadoop
This is a HDFS test file.
Big Data Analytics Lab
```

Output of Task 2 - adding files and directories

```
student@ubuntu: ~

$ hdfs dfs -mkdir -p /user/student/bdalab/data

$ hdfs dfs -put ~/sample.txt /user/student/bdalab/data

$ hdfs dfs -ls /user/student/bdalab/data
Found 1 items
-rw-r--r--   1 student supergroup   62 2026-06-25 10:30 /user/student/bdalab/data/sample.txt
```

Output of Task 3 - retrieving files

```
student@ubuntu: ~

$ hdfs dfs -ls /user/student/bdalab/data
Found 1 items
-rw-r--r--   1 student supergroup   62 2026-06-25 10:30 /user/student/bdalab/data/sample.txt

$ hdfs dfs -cat /user/student/bdalab/data/sample.txt
Hello Hadoop
This is a HDFS test file.
Big Data Analytics Lab

$ hdfs dfs -get /user/student/bdalab/data/sample.txt ~/retrieved.txt

$ cat ~/retrieved.txt
Hello Hadoop
This is a HDFS test file.
Big Data Analytics Lab
```

Output of Task 4 - deleting files and directories

```
student@ubuntu: ~

$ hdfs dfs -rm /user/student/bdalab/data/sample.txt
Deleted /user/student/bdalab/data/sample.txt

$ hdfs dfs -rm -r /user/student/bdalab
Deleted /user/student/bdalab

$ hdfs dfs -ls /user/student
$
```

The empty result of the final -ls (no items listed) confirms that the file and the directory were successfully deleted from HDFS.

Conclusion:

Basic HDFS file management was performed successfully. Directories and a file were added to HDFS, the file was retrieved by listing, viewing and downloading it, and finally the file and directory were deleted - confirming an understanding of the core hdfs dfs commands for adding, retrieving and deleting files and directories.

LAB#3

Title: "Experiment 2.2: Programmatic HDFS Management using Python and WebHDFS"

Aim:

A company stores daily activity logs from three departments (sales, hr and tech) as plain text files on a local machine. You are asked to build a small Python-based data pipeline that moves these logs into HDFS, analyses what has been stored, and then cleans up the unwanted files - all without typing a single hdfs dfs command by hand.

To perform programmatic file management on HDFS using the Python 'hdfs' library (which communicates with Hadoop through the WebHDFS REST interface), by completing the following three sub-tasks:

1. Task A - Bulk Ingestion: Write a Python program that reads all .log files from a local folder and uploads each one into an HDFS directory tree organised automatically by department name.
2. Task B - Inspection and Reporting: Write a Python program that walks the entire HDFS directory tree, reads each file's metadata (size), reads file content directly from HDFS, computes a department-wise size summary, and writes the summary report back into HDFS as a new file.
3. Task C - Selective Retrieval and Cleanup: Write a Python program that finds the largest file in HDFS and downloads it to the local machine, then deletes every file smaller than a given size threshold, and finally removes any department folder left empty by those deletions.

Theory:

Why use Python instead of the shell?

In Experiment#2.1 we managed HDFS by typing hdfs dfs commands one at a time. That is fine for a handful of files, but real big data work involves hundreds or thousands of files, and decisions that depend on the data itself - for example, 'upload each file into a folder named after its department', or 'delete every file below 200 bytes'. Such conditional, repetitive work is exactly what a programming language is for. Writing the same logic in Python makes it repeatable, automatic and error-free.

What is WebHDFS?

Hadoop exposes HDFS over the web through a REST interface called WebHDFS. This means any program that can make HTTP requests can create folders, upload files, read files and delete files in HDFS. The NameNode serves WebHDFS on port 9870 - the same port as the web dashboard we opened in Experiment#1.

The Python library named 'hdfs' is a friendly wrapper around this REST interface. Instead of building HTTP requests ourselves, we create a client object and call simple methods on it.

The main methods of the Python hdfs client

`InsecureClient(url, user)` - connects to the NameNode as a given user. It is called 'Insecure' because our cluster has no Kerberos security enabled, which is normal for a learning setup.

`client.makedirs(path)` - creates a directory in HDFS, including any missing parent directories.

`client.upload(hdfs_path, local_path)` - copies a file from the local machine into HDFS.

`client.walk(path)` - travels through the whole directory tree, giving back each folder along with the sub-folders and files inside it. It works just like Python's os.walk().

`client.status(path)` - returns a dictionary of metadata about a file or folder, including its size (the 'length' key) and its type (FILE or DIRECTORY).

`client.read(path)` and `client.write(path)` - open an HDFS file for reading or writing directly, without downloading it first.

`client.download(hdfs_path, local_path)` - copies a file from HDFS back to the local machine.

`client.delete(path, recursive=True)` - removes a file, or a folder together with everything inside it.

Purpose of each sub-task

Task A (Bulk Ingestion): Teaches automated data loading. Rather than uploading one file at a time, the program loops over every local file, decides where it belongs by looking at its name, creates the correct HDFS folder, and uploads it. Using overwrite=True and deleting any old copy first makes the script idempotent - meaning it can be run many times safely and always produces the same result.

Task B (Inspection and Reporting): Teaches how to read metadata and content from HDFS programmatically. Walking the tree shows how to discover files without knowing their names in advance. Summing sizes per department shows a simple aggregation - the same idea that MapReduce will scale up in later experiments. Writing the report back into HDFS demonstrates that a program can both consume and produce HDFS data.

Task C (Selective Retrieval and Cleanup): Teaches conditional file management. Choosing the largest file requires comparing metadata; deleting files below a threshold requires a decision per file; removing folders that have become empty requires checking a folder's contents after the deletions. Together these mirror the routine housekeeping done on a real cluster.

What is new compared to Experiment#1

Experiment#1 installed Hadoop and Java only. For this problem we additionally need Python's package installer and the 'hdfs' library. We install the library inside a Python virtual environment, which is an isolated folder holding its own packages. This is the recommended approach on modern Ubuntu, because newer versions protect the system-wide Python installation and will refuse a plain pip install.

Algorithm:

The high-level steps to complete this experiment are:

1. Start the Hadoop daemons and confirm that WebHDFS is reachable on port 9870.
2. Install python3-pip and python3-venv, create a virtual environment, and install the 'hdfs' Python library inside it.
3. Generate a set of sample .log files in a local folder, named as department_number.log.
4. Task A: Connect to HDFS, delete any old working directory, then loop over the local .log files, derive the department from each file name, create the matching HDFS folder, and upload the file.
5. Task B: Walk the HDFS tree, read each file's size from its metadata, accumulate a department-wise total, read the first line of one file to prove direct reading works, build a summary report, and write that report back into HDFS.

6. Task C: Walk the tree again to collect file sizes; find and download the largest file to the local machine; delete every file smaller than the threshold; then delete any department folder that is now empty; finally list what remains.
7. Verify the final state of HDFS using the hdfs dfs -ls -R command.

Code:

Run the steps below in order in the Ubuntu terminal. Lines starting with # are comments and need not be typed. The user is assumed to be named 'student' - replace it with your own username where it appears.

Step 0 - Prerequisites (new installations for this experiment)

0.1 Start Hadoop and confirm WebHDFS is reachable

WebHDFS is switched on by default in Hadoop 3, so no configuration change is normally needed. The curl command below asks the NameNode for the status of the root directory; a reply in JSON proves WebHDFS is working.

```
# Start the Hadoop daemons (from Experiment 1)
start-dfs.sh
start-yarn.sh

# Confirm the NameNode and DataNode are running
jps

# Ask WebHDFS for the status of the root directory '/'
curl -s "http://localhost:9870/webhdfs/v1/?op=GETFILESTATUS&user.name=student"
```

If this returns an error, open $HADOOP_HOME/etc/hadoop/hdfs-site.xml, add a property named `dfs.webhdfs.enabled` with the value `true`, then restart the daemons with stop-dfs.sh followed by start-dfs.sh.

0.2 Install the Python hdfs library in a virtual environment

A virtual environment is a private folder that holds Python packages for this project only. We create one called bdaenv, activate it, and install the library inside it. The environment must be active (you will see '(bdaenv)' in the prompt) whenever the Python scripts are run.

```
# Install Python's package installer and virtual-environment tool
sudo apt update
sudo apt install -y python3-pip python3-venv

# Create a virtual environment named 'bdaenv' in the home folder
python3 -m venv ~/bdaenv

# Activate it (the prompt will now begin with '(bdaenv)')
source ~/bdaenv/bin/activate

# Install the Python library that talks to HDFS over WebHDFS
pip install hdfs

# Confirm the library imports correctly
python3 -c "import hdfs; print('hdfs library ready')"
```

0.3 Create the sample local log files

These loops create six log files of different sizes across four departments. Their names follow the pattern department_number.log, which Task A relies on to decide the destination folder.

```
# Create a fresh local folder to hold the sample logs
```

```
rm -rf ~/bda_local && mkdir ~/bda_local && cd ~/bda_local

# A large sales log (12 lines)
for i in $(seq 1 12); do
  echo "sales record line $i : order id $((1000+i)) amount $((i*37)).50 status COMPLETED" >>
sales_01.log
done

# A small sales log (3 lines)
for i in $(seq 1 3); do
  echo "sales record line $i : order id $((2000+i)) amount $((i*11)).00 status PENDING" >>
sales_02.log
done

# A medium hr log (8 lines)
for i in $(seq 1 8); do
  echo "hr record line $i : employee E$((100+i)) department HR attendance PRESENT" >> hr_01.log
done

# A tiny hr log (2 lines)
for i in $(seq 1 2); do
  echo "hr record line $i : employee E$((200+i)) leave APPROVED" >> hr_02.log
done

# A medium tech log (6 lines)
for i in $(seq 1 6); do
  echo "tech record line $i : server node$i cpu $((40+i))% memory $((50+i))% status OK" >>
tech_01.log
done

# A tiny temp log (2 lines) - this department will become empty later
for i in $(seq 1 2); do
  echo "temp record line $i : sensor T$i reading OK" >> temp_01.log
done

# Show the sizes of the files we created (in bytes)
wc -c *.log
cd ~
```

Task A - Bulk ingestion of local log files into HDFS

Save the following program as ~/task_a.py. It connects to HDFS, clears any previous run, then loops over every local .log file. For each file it takes the part of the name before the underscore as the department, creates the matching HDFS folder, and uploads the file into it.

```
#!/usr/bin/env python3
"""Task A: Bulk-ingest local log files into an organised HDFS directory tree."""

import os
from hdfs import InsecureClient

# ---- Settings -----------------------------------------------------
HDFS_URL  = "http://localhost:9870"            # NameNode WebHDFS address
HDFS_USER = "student"                          # HDFS user name
LOCAL_DIR = os.path.expanduser("~/bda_local")  # folder holding local logs
HDFS_BASE = "/user/student/bdalab2"            # our working area in HDFS

# ---- Connect to HDFS ----------------------------------------------
client = InsecureClient(HDFS_URL, user=HDFS_USER)
print("Connected to HDFS at", HDFS_URL)

# ---- Make a clean start -------------------------------------------
# status(..., strict=False) returns None instead of raising an error
# if the path does not exist. So this safely removes any previous run.
```

```
if client.status(HDFS_BASE, strict=False) is not None:
    client.delete(HDFS_BASE, recursive=True)
    print("Removed old", HDFS_BASE)

# ---- Upload every .log file into a folder named after its department -
uploaded = 0
for filename in sorted(os.listdir(LOCAL_DIR)):
    if not filename.endswith(".log"):
        continue                      # ignore anything that is not a log

    # 'sales_01.log' -> split at '_' -> department is 'sales'
    department = filename.split("_")[0]

    # Build the destination folder, e.g. /user/student/bdalab2/sales
    hdfs_dir = HDFS_BASE + "/" + department
    client.makedirs(hdfs_dir)         # creates parent folders too

    local_path = os.path.join(LOCAL_DIR, filename)
    hdfs_path  = hdfs_dir + "/" + filename

    # overwrite=True means re-running the script will not crash
    client.upload(hdfs_path, local_path, overwrite=True)
    print("Uploaded", filename, "->", hdfs_path)
    uploaded += 1

print("\nTotal files uploaded:", uploaded)
```

Run it with:

```
source ~/bdaenv/bin/activate     # if not already active
python3 ~/task_a.py
```

Task B - Inspection and reporting

Save the following program as ~/task_b.py. It walks the HDFS tree, collects the size of every .log file, groups those sizes by department, reads the first line of one file straight out of HDFS, then writes a summary report back into HDFS.

```
#!/usr/bin/env python3
"""Task B: Walk the HDFS tree, inspect metadata, and build a summary report."""

from hdfs import InsecureClient

HDFS_URL    = "http://localhost:9870"
HDFS_USER   = "student"
HDFS_BASE   = "/user/student/bdalab2"
REPORT_PATH = HDFS_BASE + "/report.txt"

client = InsecureClient(HDFS_URL, user=HDFS_USER)

# ---- Walk the whole HDFS tree ----------------------------------------
# client.walk() works like Python's os.walk(): for each folder it returns
# the folder path, the sub-folders inside it, and the files inside it.
files       = []   # list of (full path, size in bytes)
dept_totals = {}   # department name -> total bytes

for hdfs_dir, subdirs, filenames in client.walk(HDFS_BASE):
    for name in filenames:
        if not name.endswith(".log"):
            continue                # skip the report file itself

        full_path = hdfs_dir + "/" + name
```

```
        # status() returns a dictionary of metadata about the file.
        # The 'length' key holds the file size in bytes.
        size = client.status(full_path)["length"]
        files.append((full_path, size))

        # The department is the last part of the folder path
        department = hdfs_dir.rstrip("/").split("/")[-1]
        dept_totals[department] = dept_totals.get(department, 0) + size

# ---- Print a per-file listing ----------------------------------------
print("{:<48} {:>9}".format("HDFS FILE", "SIZE (B)"))
print("-" * 58)
for path, size in sorted(files):
    print("{:<48} {:>9}".format(path, size))

# ---- Read one file directly from HDFS (no download needed) -----------
first_file = sorted(files)[0][0]
with client.read(first_file, encoding="utf-8") as reader:
    first_line = reader.readline().strip()
print("\nFirst line of", first_file)
print("  ->", first_line)

# ---- Build the summary text ------------------------------------------
total_bytes = sum(size for _, size in files)
lines = ["HDFS SUMMARY REPORT", "=" * 30, ""]
for dept in sorted(dept_totals):
    lines.append("{:<10} {:>8} bytes".format(dept, dept_totals[dept]))
lines.append("")
lines.append("Total files : {}".format(len(files)))
lines.append("Total size  : {} bytes".format(total_bytes))
report_text = "\n".join(lines) + "\n"

# ---- Write the report straight into HDFS -----------------------------
with client.write(REPORT_PATH, encoding="utf-8", overwrite=True) as writer:
    writer.write(report_text)

print("\nReport written to", REPORT_PATH)
print("-" * 30)
print(report_text)
```

Run it with:

```
python3 ~/task_b.py
```

Task C - Selective retrieval and cleanup

Save the following program as ~/task_c.py. It downloads the largest file, deletes every file below the size threshold, and then removes any department folder left with no files inside it.

```
#!/usr/bin/env python3
"""Task C: Retrieve the largest file, then selectively clean up HDFS."""

import os
from hdfs import InsecureClient

HDFS_URL       = "http://localhost:9870"
HDFS_USER      = "student"
HDFS_BASE      = "/user/student/bdalab2"
LOCAL_OUT      = os.path.expanduser("~/bda_retrieved")
SIZE_THRESHOLD = 200        # bytes: files smaller than this get deleted

client = InsecureClient(HDFS_URL, user=HDFS_USER)

# ---- Collect every .log file together with its size ------------------
log_files = []
```

```
for hdfs_dir, subdirs, filenames in client.walk(HDFS_BASE):
    for name in filenames:
        if name.endswith(".log"):
            full_path = hdfs_dir + "/" + name
            size = client.status(full_path)["length"]
            log_files.append((full_path, size))

if not log_files:
    print("No .log files found. Run Task A first.")
    raise SystemExit(0)

# ---- 1) Download the LARGEST file back to the local machine -----------
# max() with key=... picks the tuple having the biggest size
largest_path, largest_size = max(log_files, key=lambda item: item[1])

os.makedirs(LOCAL_OUT, exist_ok=True)    # create the local folder if needed
local_copy = client.download(largest_path, LOCAL_OUT, overwrite=True)

print("Largest file :", largest_path, "(", largest_size, "bytes )")
print("Downloaded to:", local_copy)

# ---- 2) Delete every file SMALLER than the threshold ------------------
print("\nDeleting files smaller than", SIZE_THRESHOLD, "bytes:")
deleted = 0
for path, size in sorted(log_files):
    if size < SIZE_THRESHOLD:
        client.delete(path)
        print("  deleted", path, "(", size, "bytes )")
        deleted += 1
print("Files deleted:", deleted)

# ---- 3) Remove any department folder that is now empty ----------------
print("\nRemoving empty department folders:")
removed_dirs = 0
for entry in client.list(HDFS_BASE):
    entry_path = HDFS_BASE + "/" + entry
    info = client.status(entry_path)
    # Only folders can be empty; list() returns [] when nothing is inside
    if info["type"] == "DIRECTORY" and len(client.list(entry_path)) == 0:
        client.delete(entry_path, recursive=True)
        print("  removed empty folder", entry_path)
        removed_dirs += 1
print("Folders removed:", removed_dirs)

# ---- 4) Show what survives --------------------------------------------
print("\nRemaining contents of", HDFS_BASE, ":")
for hdfs_dir, subdirs, filenames in client.walk(HDFS_BASE):
    for name in filenames:
        print("  ", hdfs_dir + "/" + name)
```

Run it, then verify independently with the shell command:

```
python3 ~/task_c.py

# Cross-check the final state of HDFS using the shell (-R = recursive)
hdfs dfs -ls -R /user/student/bdalab2

# Confirm the downloaded file exists locally
ls -l ~/bda_retrieved
```

Output:

The following are sample outputs (representative terminal screenshots) obtained when each script is run successfully. Timestamps and process IDs will differ from machine to machine; the file sizes shown correspond to the sample log files created in Step 0.3.

Output of Step 0.1 - WebHDFS check

```
student@ubuntu: ~

$ jps
4521 NameNode
4655 DataNode
4892 SecondaryNameNode
5320 ResourceManager
5470 NodeManager
5612 Jps

$ curl -s "http://localhost:9870/webhdfs/v1/?op=GETFILESTATUS&user.name=student"
{"FileStatus":{"accessTime":0,"blockSize":0,"childrenNum":1,"fileId":16385,
"group":"supergroup","length":0,"owner":"student","pathSuffix":"",
"permission":"755","replication":0,"type":"DIRECTORY"}}
```

Output of Step 0.2 - installing the hdfs library

```
student@ubuntu: ~

$ python3 -m venv ~/bdaenv
$ source ~/bdaenv/bin/activate
(bdaenv) $ pip install hdfs
Collecting hdfs
  Downloading hdfs-2.7.3-py3-none-any.whl (34 kB)
Installing collected packages: docopt, six, requests, hdfs
Successfully installed hdfs-2.7.3 docopt-0.6.2 requests-2.32.3 six-1.16.0

(bdaenv) $ python3 -c "import hdfs; print('hdfs library ready')"
hdfs library ready
```

Output of Step 0.3 - sample log files created

```
student@ubuntu: ~/bda_local

(bdaenv) $ wc -c *.log
  528 hr_01.log
   96 hr_02.log
  805 sales_01.log
  192 sales_02.log
  378 tech_01.log
   84 temp_01.log
 2083 total
```

Output of Task A - bulk ingestion

```
student@ubuntu: ~

(bdaenv) $ python3 ~/task_a.py
Connected to HDFS at http://localhost:9870
Uploaded hr_01.log -> /user/student/bdalab2/hr/hr_01.log
Uploaded hr_02.log -> /user/student/bdalab2/hr/hr_02.log
Uploaded sales_01.log -> /user/student/bdalab2/sales/sales_01.log
Uploaded sales_02.log -> /user/student/bdalab2/sales/sales_02.log
Uploaded tech_01.log -> /user/student/bdalab2/tech/tech_01.log
Uploaded temp_01.log -> /user/student/bdalab2/temp/temp_01.log

Total files uploaded: 6
```

Output of Task B - inspection and reporting

```
student@ubuntu: ~

(bdaenv) $ python3 ~/task_b.py
HDFS FILE                                          SIZE (B)
------------------------------------------------------------
/user/student/bdalab2/hr/hr_01.log                      528
/user/student/bdalab2/hr/hr_02.log                       96
/user/student/bdalab2/sales/sales_01.log                805
/user/student/bdalab2/sales/sales_02.log                192
/user/student/bdalab2/tech/tech_01.log                  378
/user/student/bdalab2/temp/temp_01.log                   84

First line of /user/student/bdalab2/hr/hr_01.log
  -> hr record line 1 : employee E101 department HR attendance PRESENT

Report written to /user/student/bdalab2/report.txt
------------------------------
HDFS SUMMARY REPORT
==============================

hr              624 bytes
sales           997 bytes
tech            378 bytes
temp             84 bytes

Total files : 6
Total size  : 2083 bytes
```

Output of Task C - selective retrieval and cleanup

```
student@ubuntu: ~

(bdaenv) $ python3 ~/task_c.py
Largest file : /user/student/bdalab2/sales/sales_01.log ( 805 bytes )
Downloaded to: /home/student/bda_retrieved/sales_01.log

Deleting files smaller than 200 bytes:
  deleted /user/student/bdalab2/hr/hr_02.log ( 96 bytes )
  deleted /user/student/bdalab2/sales/sales_02.log ( 192 bytes )
  deleted /user/student/bdalab2/temp/temp_01.log ( 84 bytes )
Files deleted: 3

Removing empty department folders:
  removed empty folder /user/student/bdalab2/temp
Folders removed: 1

Remaining contents of /user/student/bdalab2 :
   /user/student/bdalab2/report.txt
   /user/student/bdalab2/hr/hr_01.log
   /user/student/bdalab2/sales/sales_01.log
   /user/student/bdalab2/tech/tech_01.log
```

Output - independent verification using the shell

```
student@ubuntu: ~

(bdaenv) $ hdfs dfs -ls -R /user/student/bdalab2
drwxr-xr-x   - student supergroup    0 2026-06-25 11:20 /user/student/bdalab2/hr
-rw-r--r--   1 student supergroup  528 2026-06-25 11:18 /user/student/bdalab2/hr/hr_01.log
-rw-r--r--   1 student supergroup  151 2026-06-25 11:19 /user/student/bdalab2/report.txt
drwxr-xr-x   - student supergroup    0 2026-06-25 11:20 /user/student/bdalab2/sales
-rw-r--r--   1 student supergroup  805 2026-06-25 11:18 /user/student/bdalab2/sales/sales_01.log
drwxr-xr-x   - student supergroup    0 2026-06-25 11:18 /user/student/bdalab2/tech
-rw-r--r--   1 student supergroup  378 2026-06-25 11:18 /user/student/bdalab2/tech/tech_01.log

(bdaenv) $ ls -l ~/bda_retrieved
total 4
-rw-rw-r-- 1 student student 805 Jun 25 11:20 sales_01.log
```

The final listing confirms the expected result: the 'temp' folder has disappeared because its only file was below the threshold, the three small files are gone, the three large files survive, and the report generated by Task B is present in HDFS while the largest file has been retrieved onto the local machine.

Conclusion:

Programmatic HDFS file management was performed successfully using Python and the WebHDFS interface. Local log files were bulk-ingested into an automatically organised HDFS directory tree (Task A); the tree was walked, metadata and content were read, and a department-wise summary report was computed and written back into HDFS (Task B); and the largest file was retrieved to the local machine while undersized files and the resulting empty directory were removed (Task C). The final state of HDFS was independently verified using hdfs dfs -ls -R.

LAB#4

Title: "Experiment 3.1: Understanding the MapReduce Paradigm using Hadoop Streaming"

Aim:

To understand the MapReduce paradigm by writing, in Python using Hadoop Streaming, a program that counts how many times each word appears in a text file. The tasks are:

1. Write a mapper that emits each word with the value 1.
2. Write a reducer that adds up the 1s for each word to give its total count.
3. Test the mapper and reducer locally using a Unix pipe.
4. Run the same mapper and reducer as a real MapReduce job on Hadoop and read the result from HDFS.

Theory:

What is MapReduce?:

MapReduce is the original way of processing large data in Hadoop. Its big idea is simple: instead of processing a huge file on one machine, we break the work into many small independent pieces, do them in parallel, and then combine the results. Every MapReduce job is expressed in just two functions that the programmer writes:

The Map function: runs once on each small piece of the input and emits a set of intermediate (key, value) pairs. It looks at one record at a time and does not care about the rest of the data.

The Reduce function: receives all the values that share the same key, grouped together, and combines them into a final result (for example, a sum or a count).

Between Map and Reduce there is an automatic stage called the Shuffle and Sort. Hadoop performs it for us: it collects every pair produced by the mappers, sorts them by key, and groups all values for the same key so they arrive at the reducer together. The programmer never writes this part - but understanding that it happens is the key to understanding MapReduce.

The three phases in one line:

MAP (split the work) → **SHUFFLE & SORT** (group by key, done by Hadoop) → **REDUCE** (combine each group).

What is Hadoop Streaming, and why we use Python with it

Hadoop was written in Java, so MapReduce programs are traditionally written in Java too. Hadoop Streaming is a built-in tool that removes this restriction: it lets us write the mapper and reducer as ordinary programs in ANY language - here, Python - as long as they follow one simple rule:

The mapper reads input line by line from standard input (stdin) and writes (key, value) pairs to standard output (stdout), separated by a TAB. The reducer reads those pairs from stdin (already sorted by key) and writes the final result to stdout.

This means we can test the whole logic on the local machine without Hadoop at all, using a normal Unix pipe:

```
cat input.txt | python3 mapper.py | sort | python3 reducer.py
```

The 'sort' command in the middle imitates exactly what Hadoop's Shuffle and Sort phase does. Once the logic works locally, the very same two scripts are handed to Hadoop to run at scale across the cluster. This is why Streaming is perfect for a first MapReduce lab: the paradigm is fully visible and the language is friendly.

One important rule about the reducer:

Because Hadoop sorts the pairs by key before the reducer sees them, all the values for one key arrive together, one after another. The reducer therefore keeps track of the 'current key': while the incoming key stays the same it keeps accumulating; the moment the key changes, it prints the finished result for the previous key and starts fresh. This single pattern is used in both problems in Experiment#3.

Theoretical Background:

Word Count is called the 'Hello World' of Big Data because it is the smallest problem that still needs both a Map and a Reduce phase, so it shows the whole paradigm with no distracting logic.

The Map phase: For every line of text, the mapper breaks the line into words and, for each word, outputs the pair (word, 1). The number 1 simply means 'I have seen this word once'. The mapper makes no attempt to count - it just reports one sighting at a time.

The Shuffle and Sort: Hadoop groups all the pairs by word, so that every 1 belonging to the same word lines up together, for example: big 1, big 1, big 1.

The Reduce phase: For each word, the reducer adds up all the 1s and outputs the final total, for example big 3. This is where the actual counting happens.

The purpose of testing locally first (Task 3) is to confirm the logic is correct in seconds, before spending time submitting a full Hadoop job. Running it on Hadoop afterwards (Task 4) proves the identical code scales to the real distributed engine.

Algorithm:

The high-level steps to complete this experiment are:

1. Mapper: read each line from stdin; convert to lower case; split it into words; for each word, print 'word <TAB> 1'.
2. Reducer: read 'word <TAB> 1' lines (which Hadoop has sorted by word); keep a running total for the current word; when the word changes, print the previous word and its total; print the last word at the end.
3. Test locally with: cat input.txt | python3 mapper.py | sort | python3 reducer.py.
4. Put the input file into HDFS, then run the two scripts on Hadoop using the Hadoop Streaming jar.
5. View the output produced in the HDFS output directory.

Code:

Create a working folder and put the two scripts and a sample input file inside it.

```
# Make sure Hadoop is running
start-dfs.sh && start-yarn.sh

# Create a folder for this problem
mkdir -p ~/wordcount && cd ~/wordcount
```

Task 1 - The Mapper (mapper.py)

Save this as ~/wordcount/mapper.py. It reads text line by line and prints every word followed by a TAB and the number 1.

```
#!/usr/bin/env python3
import sys
import re
```

```
# Read the input one line at a time from standard input
for line in sys.stdin:
    # Make everything lower case so 'Big' and 'big' count as the same word
    line = line.strip().lower()

    # findall pulls out runs of letters/numbers, ignoring spaces and punctuation
    for word in re.findall(r"[a-z0-9]+", line):
        # Emit the pair: word <TAB> 1
        print(f"{word}\t1")
```

Task 2 - The Reducer (reducer.py)

Save this as ~/wordcount/reducer.py. It receives the pairs sorted by word and adds up the 1s for each word.

```
#!/usr/bin/env python3
import sys

current_word  = None   # the word we are currently counting
current_count = 0      # its running total

# Input arrives already sorted by word, so equal words are adjacent
for line in sys.stdin:
    word, count = line.strip().split("\t")
    count = int(count)

    if word == current_word:
        # Same word as before -> keep adding
        current_count += count
    else:
        # The word changed -> print the finished total for the old word
        if current_word is not None:
            print(f"{current_word}\t{current_count}")
        # Start counting the new word
        current_word  = word
        current_count = count

# Don't forget the very last word in the file
if current_word is not None:
    print(f"{current_word}\t{current_count}")
```

Task 3 - Create sample input and test locally

Create a small text file, then test the logic with a plain Unix pipe. The 'sort' in the middle does the same job as Hadoop's Shuffle and Sort.

```
# Create a small sample text file
cat > input.txt << 'EOF'
Big data is big
Hadoop processes big data
data data data
EOF

# Test the complete MapReduce logic locally (no Hadoop needed yet)
cat input.txt | python3 mapper.py | sort | python3 reducer.py
```

Task 4 - Run as a real MapReduce job on Hadoop

Now we upload the input to HDFS and run the identical scripts on the cluster using the Hadoop Streaming jar. The -files option ships our two Python scripts to the worker nodes.

```
# Create an input folder in HDFS and upload the text file
hdfs dfs -mkdir -p /user/student/wc/input
hdfs dfs -put -f input.txt /user/student/wc/input

# Remove any old output folder (Hadoop refuses to overwrite it)
```

```
hdfs dfs -rm -r -f /user/student/wc/output

# Run the MapReduce job using Hadoop Streaming
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -files mapper.py,reducer.py \
  -mapper "python3 mapper.py" \
  -reducer "python3 reducer.py" \
  -input  /user/student/wc/input \
  -output /user/student/wc/output

# View the result (part-00000 is the reducer's output file)
hdfs dfs -cat /user/student/wc/output/part-00000
```

Output

Sample outputs (representative terminal screenshots). Job IDs and counters will differ from machine to machine.

Local test (Task 3)

```
student@ubuntu: ~/wordcount

$ cat input.txt | python3 mapper.py | sort | python3 reducer.py
big     3
data    5
hadoop  1
is      1
processes       1
```

Hadoop job run and result (Task 4)

```
student@ubuntu: ~/wordcount

$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
    -files mapper.py,reducer.py -mapper "python3 mapper.py" \
    -reducer "python3 reducer.py" -input /user/student/wc/input \
    -output /user/student/wc/output
packageJobJar: [] ...
INFO mapreduce.Job: Running job: job_1730880000000_0001
INFO mapreduce.Job:  map 0% reduce 0%
INFO mapreduce.Job:  map 100% reduce 100%
INFO mapreduce.Job: Job job_1730880000000_0001 completed successfully
        Map input records=3
        Reduce output records=5

$ hdfs dfs -cat /user/student/wc/output/part-00000
big     3
data    5
hadoop  1
is      1
processes       1
```

The count matches the local test, confirming that the same Python mapper and reducer produced the correct word counts when run as a genuine MapReduce job on Hadoop.

LAB#5

Title: "Experiment 3.2: Sales Aggregation using MapReduce"

Aim:

To take the MapReduce paradigm one step further by processing structured (CSV) data and computing more than a simple count. Using Hadoop Streaming in Python, analyse a sales file and, for each product category, compute the total sales amount, the number of transactions, and the average sale value. The tasks are:

1. Write a mapper that reads a CSV sales record and emits (category, amount).
2. Write a reducer that, for each category, computes the total amount, the transaction count, and the average.
3. Test locally with a Unix pipe.
4. Run the job on Hadoop and read the result from HDFS.

Theory:

Word Count only ever emitted the constant value 1 and only ever added up counts. This problem is one level higher for three reasons, and each reason teaches something new about MapReduce:

1. The input is structured. Each line is a CSV record with three fields (date, category, amount). The mapper must split the line on commas and pick out the fields it needs, rather than just splitting on spaces. It must also quietly skip the header row and any broken lines.

2. The value carries real data. Instead of emitting 1, the mapper emits the actual sale amount. The chosen key is the category, because the category is what we want to group by. Choosing the right key is the central design decision in every MapReduce job.

3. The reducer computes a derived metric. For each category the reducer must keep two running quantities at once - a sum of amounts and a count of transactions - and from them compute the average (sum divided by count). Producing an average is a classic example that a count alone cannot give.

The purpose of skipping the header and malformed rows inside the mapper (rather than cleaning the file beforehand) is to show that a mapper must be defensive: in real data some records are always dirty, and the job should not crash because of them.

Algorithm:

1. Mapper: for each line, split on commas; ignore the header and any line that does not have exactly three fields or whose amount is not a number; emit 'category <TAB> amount'.
2. Reducer: for the sorted input, keep a running sum and count for the current category; when the category changes, print category, total, count and average for the previous category; print the last category at the end.
3. Test locally with a Unix pipe.
4. Upload the CSV to HDFS and run the job with the Hadoop Streaming jar; view the output.

Code:

```
# Create a folder for this problem
mkdir -p ~/sales && cd ~/sales
```

Task 1 - The Mapper (mapper.py)

Save this as ~/sales/mapper.py. It parses each CSV line and outputs the category with its sale amount. The header row is skipped automatically because its amount field ('amount') is not a number.

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue                    # skip blank lines

    parts = line.split(",")         # CSV -> list of fields
    if len(parts) != 3:             # skip malformed rows
        continue

    date, category, amount = parts
    try:
        amount = float(amount)      # the header 'amount' fails here
    except ValueError:              # ... and is skipped
        continue

    # Key = category, Value = amount
    print(f"{category}\t{amount}")
```

Task 2 - The Reducer (reducer.py)

Save this as ~/sales/reducer.py. For each category it keeps a running total and count, and prints the total, count and average when the category changes.

```
#!/usr/bin/env python3
import sys

current = None      # category currently being summed
total   = 0.0       # running total of amounts
count   = 0         # running number of transactions

def emit(category, total, count):
    average = total / count if count else 0
    # category <TAB> total <TAB> count <TAB> average
    print(f"{category}\t{total:.2f}\t{count}\t{average:.2f}")

for line in sys.stdin:
    category, amount = line.strip().split("\t")
    amount = float(amount)

    if category == current:
        total += amount             # same category -> accumulate
        count += 1
    else:
        if current is not None:
            emit(current, total, count)   # finish the previous category
        current = category          # start the new category
        total   = amount
        count   = 1

# Emit the final category
if current is not None:
    emit(current, total, count)
```

Task 3 - Create sample data and test locally

```
# Create a small CSV file with a header row
cat > sales.csv << 'EOF'
date,category,amount
2026-01-01,Electronics,250.00
2026-01-01,Grocery,45.50
2026-01-02,Electronics,999.99
2026-01-02,Clothing,120.00
2026-01-03,Grocery,30.00
2026-01-03,Grocery,75.25
2026-01-04,Clothing,60.00
2026-01-04,Electronics,150.00
2026-01-05,Grocery,20.00
EOF

# Test locally: output is  category <TAB> total <TAB> count <TAB> average
cat sales.csv | python3 mapper.py | sort | python3 reducer.py
```

Task 4 - Run as a MapReduce job on Hadoop

```
# Upload the CSV into HDFS
hdfs dfs -mkdir -p /user/student/sales/input
hdfs dfs -put -f sales.csv /user/student/sales/input

# Remove any previous output
hdfs dfs -rm -r -f /user/student/sales/output

# Run the job with Hadoop Streaming
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -files mapper.py,reducer.py \
  -mapper  "python3 mapper.py" \
  -reducer "python3 reducer.py" \
  -input   /user/student/sales/input \
  -output  /user/student/sales/output

# View the result
hdfs dfs -cat /user/student/sales/output/part-00000
```

Output:

Sample outputs (representative terminal screenshots). The four columns are category, total, count and average.

Local test (Task 3)

```
student@ubuntu: ~/sales

$ cat sales.csv | python3 mapper.py | sort | python3 reducer.py
Clothing 180.00   2        90.00
Electronics       1399.99  3        466.66
Grocery  170.75   4        42.69
```

Hadoop job result (Task 4)

```
student@ubuntu: ~/sales

$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
    -files mapper.py,reducer.py -mapper "python3 mapper.py" \
    -reducer "python3 reducer.py" -input /user/student/sales/input \
    -output /user/student/sales/output
INFO mapreduce.Job: Running job: job_1730880000000_0002
INFO mapreduce.Job:  map 100% reduce 100%
INFO mapreduce.Job: Job job_1730880000000_0002 completed successfully
        Map input records=10
        Reduce output records=3


$ hdfs dfs -cat /user/student/sales/output/part-00000
Clothing 180.00   2        90.00
Electronics       1399.99  3        466.66
Grocery  170.75   4        42.69
```

Reading the result: Electronics had 3 transactions totalling 1399.99 with an average sale of 466.66; Grocery had the most transactions (4) but the smallest average (42.69). Note that the 10 input records include the header line, which the mapper correctly discarded, leaving 9 real sales grouped into 3 categories.

LAB#6

Title: "Apache Hive: Databases, Tables and Queries for Data Analysis"

Aim:

To install and configure Apache Hive on top of the existing Hadoop setup, and to use HiveQL (Hive's SQL-like language) to create databases and tables, load data, and run queries for data analysis. The tasks are:

1. Install and configure Apache Hive (including the guava fix and initialising the metastore).
2. Start the Hive shell and create a database, then switch into it.
3. Create a table with a schema that matches a CSV dataset.
4. Load data into the table from a local file.
5. Run queries for data analysis: selection, filtering, aggregation (GROUP BY), and sorting.
6. Drop the table and database to clean up.

Theory:

What is Apache Hive, and why use it?

In the previous Experiment we analysed sales data by writing a mapper and a reducer by hand. That works, but writing a program for every question is slow and needs programming skill. Apache Hive solves this. Hive is a data warehouse system built on top of Hadoop that lets us ask questions about very large data using a language almost identical to SQL, called HiveQL.

The magic is that Hive automatically translates each HiveQL query into a Hadoop job (MapReduce or Tez) and runs it on the cluster. So the same 'total sales per category' result that needed a full mapper and reducer in Experiment 3 becomes a single line - SELECT category, SUM(amount) FROM sales GROUP BY category - in Hive. This is exactly why higher-level tools exist: they let analysts work with big data without writing low-level code.

HiveQL and how a query becomes a job

When you type a HiveQL query, Hive parses it, makes a plan, and for anything that needs real computation (such as GROUP BY, COUNT or SUM) it launches a Hadoop job behind the scenes. You will actually see the job's progress printed on screen - proof that Hive is powered by the same engine we used directly earlier. Simple queries that only read rows (such as SELECT * with no aggregation) are answered instantly without a job, using a quick 'fetch' operation.

The Metastore and Derby

Hive needs somewhere to remember the definitions of your databases and tables - their names, columns and data types. This information (called metadata) is kept in a small relational database called the Metastore. For a single-machine learning setup, Hive ships with an embedded database named Apache Derby, which needs no separate installation. Before first use, the Metastore must be created once using a tool called schematool.

Managed tables, the warehouse, and schema-on-read

By default Hive creates 'managed' tables, whose data is stored inside a special HDFS folder called the warehouse (at /user/hive/warehouse). When you load a file into a managed table, Hive moves that data into the warehouse; if you drop the table, the data is deleted too.

An important idea is schema-on-read. Unlike a traditional database, Hive does not check the data when it is loaded - it simply stores the file. The columns and types you declare are only applied when you read the data with a query. This

makes loading extremely fast and is well suited to big data, where checking every row up front would be too slow.

Why each task

Install and configure (Task 1): Hive is a separate download that plugs into Hadoop. Configuration involves setting environment variables, fixing a known library clash (the guava jar), preparing HDFS folders, and creating the Metastore - the one-time groundwork that lets everything else work.

Create a database (Task 2): A database in Hive is a namespace that groups related tables, keeping different projects tidy - just like a schema in ordinary SQL.

Create a table (Task 3): The table defines the structure (columns and types) that Hive will apply to our CSV data when we query it. Here we also tell Hive that fields are separated by commas and that the first line is a header to be skipped.

Load data (Task 4): This places our CSV file under the table so it can be queried. We use a local file for convenience.

Run queries (Task 5): This is the payoff - answering real analytical questions (counts, totals, averages, filters, rankings) with short SQL statements instead of code.

Algorithm:

1. Download and extract Apache Hive; set HIVE_HOME and update PATH.
2. Fix the guava library clash by replacing Hive's old guava jar with Hadoop's newer one.
3. Start Hadoop, create the HDFS warehouse and tmp directories, and give them write permission.
4. Initialise the Derby Metastore using schematool.
5. Launch the Hive shell; create a database and switch into it.
6. Create a table whose columns match the sales CSV, with comma-separated fields and a skipped header line.
7. Load the local sales.csv file into the table.
8. Run analysis queries: view all rows, count rows, total/count/average per category, filter by amount, and rank categories by total sales.
9. Drop the table and database to clean up.

Code:

The user is assumed to be 'student'. Commands that begin at the Ubuntu shell are shown in grey; HiveQL statements typed inside the Hive shell are shown in blue-green. This experiment reuses the sales.csv file from Experiment 3 - make sure it is present in the home folder (~/sales.csv).

Task 1 - Install and configure Hive

1.1 Download, extract and set environment variables

We download Hive 3.1.3 (which works with Hadoop 3.x), extract it to /usr/local/hive, and add HIVE_HOME to the environment so the 'hive' command works from anywhere.

```
# Move to the home folder and download Apache Hive 3.1.3
cd ~
wget https://archive.apache.org/dist/hive/hive-3.1.3/apache-hive-3.1.3-bin.tar.gz

# Extract it and move it to /usr/local/hive
tar -xzf apache-hive-3.1.3-bin.tar.gz
sudo mv apache-hive-3.1.3-bin /usr/local/hive

# Add Hive's environment variables to ~/.bashrc
cat >> ~/.bashrc << 'EOF'

# ----- Hive environment variables -----
export HIVE_HOME=/usr/local/hive
export PATH=$PATH:$HIVE_HOME/bin
EOF

# Reload so the new variables take effect now
source ~/.bashrc
```

1.2 Fix the guava library clash

Hadoop and Hive ship different versions of a Java library called guava, and Hive's older copy causes an error when creating the Metastore. The fix is to delete Hive's old guava jar and copy Hadoop's newer one in its place. Using a wildcard (*) means we do not need to know the exact version numbers.

```
# Remove Hive's old guava jar
rm -f $HIVE_HOME/lib/guava-*.jar

# Copy Hadoop's newer guava jar into Hive's lib folder
cp $(find $HADOOP_HOME/share/hadoop -name 'guava-*.jar' | head -1) $HIVE_HOME/lib/
```

1.3 Prepare HDFS folders and create the Metastore

Hive stores table data in an HDFS 'warehouse' folder and uses a 'tmp' folder for scratch work; both need to exist and be writable. Finally, schematool creates the Derby Metastore. Run this from the home folder so the Metastore files are always created in the same place.

```
# Make sure Hadoop is running
start-dfs.sh && start-yarn.sh

# Create the HDFS directories Hive needs, and make them writable
hdfs dfs -mkdir -p /user/hive/warehouse
hdfs dfs -mkdir -p /tmp
hdfs dfs -chmod g+w /user/hive/warehouse
hdfs dfs -chmod g+w /tmp

# Create the Derby metastore (run this ONCE, from the home folder)
cd ~
schematool -dbType derby -initSchema
```

Note: Because we use embedded Derby, always launch Hive from the same folder (the home folder ~). Derby allows only one Hive session at a time; if you see a 'metastore_db' lock error, close any other Hive session and try again.

Task 2 - Start Hive and create a database

Launch the Hive shell by typing hive. At the hive> prompt, create a database for our lab work and switch into it. (These are HiveQL statements - note that each ends with a semicolon.)

```
# Launch the Hive shell (from the home folder)
cd ~
hive
```

```
-- Create a database to hold our tables
CREATE DATABASE IF NOT EXISTS bdalab;

-- List databases to confirm it was created
SHOW DATABASES;

-- Switch into our new database
USE bdalab;
```

Task 3 - Create a table

We create a table whose three columns match the sales.csv file. ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' tells Hive the file is comma-separated. The TBLPROPERTIES line tells Hive to skip the first line, which is the CSV header.

```
-- Create a table matching the columns of sales.csv
CREATE TABLE IF NOT EXISTS sales (
    sale_date STRING,
    category  STRING,
    amount    DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
TBLPROPERTIES ('skip.header.line.count'='1');

-- Show the table's structure
DESCRIBE sales;
```

Task 4 - Load data into the table

LOAD DATA LOCAL INPATH reads a file from the local Ubuntu filesystem and places it under the table. (Without the word LOCAL, Hive would look in HDFS instead.)

```
-- Load the local sales.csv file into the sales table
LOAD DATA LOCAL INPATH '/home/student/sales.csv' INTO TABLE sales;
```

Task 5 - Run queries for data analysis

Now we answer analytical questions. Each query is explained by its comment. Aggregation queries (COUNT, SUM, GROUP BY) launch a Hadoop job; a plain SELECT * is answered instantly.

Query 1 - View all the data

```
SELECT * FROM sales;
```

Query 2 - Count the number of sales records

```
SELECT COUNT(*) AS total_records FROM sales;
```

Query 3 - Total, count and average sales per category

```
SELECT category,
       SUM(amount)          AS total_sales,
       COUNT(*)             AS num_transactions,
       ROUND(AVG(amount),2) AS average_sale
FROM sales
GROUP BY category;
```

Query 4 - Only the high-value sales (amount greater than 100)

```
SELECT * FROM sales WHERE amount > 100;
```

Query 5 - Rank the categories by total sales (highest first)

```
SELECT category, SUM(amount) AS total_sales
FROM sales
GROUP BY category
ORDER BY total_sales DESC;
```

Task 6 - Clean up

Dropping the table removes both its definition and (because it is a managed table) its data. Then we drop the database and leave the Hive shell.

```
-- Remove the table and the database
DROP TABLE IF EXISTS sales;
DROP DATABASE IF EXISTS bdalab;

-- Leave the Hive shell
EXIT;
```

Output

Sample outputs (representative terminal screenshots). Warnings, timing and job IDs vary from machine to machine; the query results correspond to the sales.csv dataset from Experiment 3.

Output of Task 1.3 - creating the Metastore

```
student@ubuntu: ~

$ schematool -dbType derby -initSchema
Metastore connection URL:  jdbc:derby:;databaseName=metastore_db;create=true
Metastore Connection Driver :  org.apache.derby.jdbc.EmbeddedDriver
Metastore connection User:  APP
Starting metastore schema initialization to 3.1.0
Initialization script hive-schema-3.1.0.derby.sql
Initialization script completed
schemaTool completed
```

Output of Task 2 - starting Hive and creating a database

```
student@ubuntu: ~

$ hive
Hive Session ID = 8f2b1c34-....
Logging initialized using configuration in jar:file:/usr/local/hive/lib/...

hive> CREATE DATABASE IF NOT EXISTS bdalab;
OK
Time taken: 0.42 seconds
hive> SHOW DATABASES;
OK
bdalab
default
Time taken: 0.06 seconds, Fetched: 2 row(s)
hive> USE bdalab;
OK
Time taken: 0.03 seconds
```

Output of Task 3 - creating the table

```
hive> (in database bdalab)

hive> CREATE TABLE IF NOT EXISTS sales ( ... );
OK
Time taken: 0.55 seconds
hive> DESCRIBE sales;
OK
sale_date            string
category             string
amount               double
Time taken: 0.09 seconds, Fetched: 3 row(s)
```

Output of Task 4 - loading the data

```
hive> (in database bdalab)

hive> LOAD DATA LOCAL INPATH '/home/student/sales.csv' INTO TABLE sales;
Loading data to table bdalab.sales
OK
Time taken: 0.63 seconds
```

Output of Query 1 - SELECT * (answered instantly, no job)

```
hive> (in database bdalab)

hive> SELECT * FROM sales;
OK
2026-01-01    Electronics    250.0
2026-01-01    Grocery        45.5
2026-01-02    Electronics    999.99
2026-01-02    Clothing       120.0
2026-01-03    Grocery        30.0
```

```
2026-01-03    Grocery        75.25
2026-01-04    Clothing       60.0
2026-01-04    Electronics    150.0
2026-01-05    Grocery        20.0
Time taken: 0.20 seconds, Fetched: 9 row(s)
```

Output of Query 2 - COUNT(*) (launches a Hadoop job)

```
hive> (in database bdalab)

hive> SELECT COUNT(*) AS total_records FROM sales;
Query ID = student_20260625_...
Total jobs = 1
Launching Job 1 out of 1
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-06-25 12:05:11  Stage-1 map = 100%,  reduce = 100%
OK
9
Time taken: 18.7 seconds, Fetched: 1 row(s)
```

Output of Query 3 - total, count and average per category

```
hive> (in database bdalab)

hive> SELECT category, SUM(amount), COUNT(*), ROUND(AVG(amount),2)
    > FROM sales GROUP BY category;
...  (Hadoop job runs)  ...
OK
Clothing       180.0     2    90.0
Electronics    1399.99   3    466.66
Grocery        170.75    4    42.69
Time taken: 21.3 seconds, Fetched: 3 row(s)
```

Output of Query 4 - high-value sales (amount > 100)

```
hive> (in database bdalab)

hive> SELECT * FROM sales WHERE amount > 100;
OK
2026-01-01    Electronics    250.0
2026-01-02    Electronics    999.99
2026-01-02    Clothing       120.0
2026-01-04    Electronics    150.0
Time taken: 0.24 seconds, Fetched: 4 row(s)
```

Output of Query 5 - categories ranked by total sales

```
hive> (in database bdalab)

hive> SELECT category, SUM(amount) AS total_sales
    > FROM sales GROUP BY category ORDER BY total_sales DESC;
...  (Hadoop job runs)  ...
OK
Electronics    1399.99
Clothing       180.0
Grocery        170.75
Time taken: 24.9 seconds, Fetched: 3 row(s)
```

Notice that Query 1 and Query 4 (plain row reads) return almost instantly, while the aggregation queries (2, 3 and 5) each launch a Hadoop job and take longer - clearly showing that Hive turns SQL into the same MapReduce processing we wrote by hand in Experiment 3.

Result:

Apache Hive was installed and configured on top of Hadoop, and the Derby Metastore was initialised successfully. A database and a table were created, the sales dataset was loaded, and a range of analytical queries - counting, per-category totals and averages, filtering and ranking - were answered using short HiveQL statements. The experiment demonstrated how Hive provides an SQL-like abstraction that is automatically executed as Hadoop jobs, making big-data analysis far simpler than writing MapReduce programs by hand.

LAB#7

Title: "MongoDB: Count, Sort, Limit, Skip & Aggregate"

Aim:

To store and query document (NoSQL) data using MongoDB from Python, exploring the document data model. The tasks are:

1. Task a - Insert sample documents into a MongoDB collection and perform count, sort, limit and skip operations to retrieve and browse the data.
2. Task b - Build an aggregation pipeline (using $match, $group and $sort) to compute summary statistics over the collection.

Theory:

What is MongoDB and how is it different?

MongoDB is a NoSQL database. Unlike Hive or a traditional SQL database, where data must fit into fixed rows and columns of a table, MongoDB stores data as documents. A document looks exactly like a Python dictionary (or a JSON object): a set of field-and-value pairs, for example {"name": "Laptop", "price": 75000}. A group of documents is called a collection, which is the rough equivalent of a table.

The big advantage of this model is flexibility: different documents in the same collection can have different fields, and values can themselves be lists or nested documents. This makes MongoDB a good fit for data that does not have a rigid, unchanging structure. This experiment is important because it introduces a completely different way of thinking about data from the table-and-SQL model of the previous experiments.

The operations we will use

count (count_documents) - counts how many documents match a condition, e.g. how many products are Electronics.

sort - orders the results by a field, ascending (1) or descending (-1), e.g. most expensive first.

limit - keeps only the first N results, e.g. the top 3.

skip - ignores the first N results. Together, skip and limit give paging: skip 3 then limit 3 returns 'page 2' of the data.

aggregate - runs a pipeline of stages that transform the data. $match filters documents (like WHERE), $group groups them and computes summaries such as counts and averages (like GROUP BY), and $sort orders the summarised result. Data flows through the stages one after another, like a factory conveyor belt.

Purpose of each task

Task a teaches the basic 'read' operations every database needs: how to browse, order and page through documents. These are the everyday queries a developer runs constantly.

Task b teaches analysis. The aggregation pipeline is MongoDB's most powerful feature - it computes the summary statistics (counts, averages, totals per group) that turn raw documents into insight, all inside the database.

Algorithm:

1. Start the MongoDB service and connect to it from Python using PyMongo.
2. Insert a list of sample product documents into a collection (clearing it first so the script is repeatable).

3. Task a: use count_documents, then find(...).sort(...).limit(...), then find(...).sort(...).skip(...).limit(...) to browse and page the data.
4. Task b: build an aggregation pipeline with $group and $sort to summarise per category, and a second pipeline with $match then $group to summarise a filtered subset.
5. Print the results returned by each query.

Code:

Dependencies to install for this experiment

```
# 1) Install the MongoDB server (Community Edition) on Ubuntu
sudo apt update
sudo apt install -y mongodb-server-core mongodb-clients   # or follow MongoDB's
                                                          # official apt repo

# Start the MongoDB service and enable it at boot
sudo systemctl start mongod
sudo systemctl enable mongod
sudo systemctl status mongod        # should say 'active (running)'

# 2) Install the Python driver inside our virtual environment
source ~/bdaenv/bin/activate
pip install pymongo
```

Note: On some Ubuntu versions the package is named `mongodb-org` and installed from MongoDB's official repository; if `apt install mongodb-server-core` is not found, follow MongoDB's official 'Install on Ubuntu' apt steps. Once `systemctl status mongod` shows 'running', the Python code below is identical either way.

Task a - Insert documents; count, sort, limit, skip

Save as ~/mongo_task_a.py.

```
#!/usr/bin/env python3
from pymongo import MongoClient

# Connect to the MongoDB server running on this machine (default port 27017)
client = MongoClient("mongodb://localhost:27017/")
db = client["bdalab"]            # database (created on first use)
products = db["products"]        # collection (like a table)

# Start clean so the script can be run many times
products.delete_many({})

# A list of sample documents (each is just a Python dictionary)
docs = [
    {"name":"Laptop",     "category":"Electronics","price":75000,"stock":12, "rating":4.5},
    {"name":"Mouse",      "category":"Electronics","price":800,  "stock":150,"rating":4.1},
    {"name":"Keyboard",   "category":"Electronics","price":1500, "stock":90, "rating":4.3},
    {"name":"Rice Bag",   "category":"Grocery",    "price":1200, "stock":40, "rating":4.0},
    {"name":"Cooking Oil","category":"Grocery",    "price":250,  "stock":200,"rating":3.9},
    {"name":"T-Shirt",    "category":"Clothing",   "price":600,  "stock":75, "rating":4.2},
    {"name":"Jeans",      "category":"Clothing",   "price":2200, "stock":30, "rating":4.4},
    {"name":"Monitor",    "category":"Electronics","price":12000,"stock":25, "rating":4.6},
    {"name":"Notebook",   "category":"Grocery",    "price":60,   "stock":300,"rating":3.7},
    {"name":"Jacket",     "category":"Clothing",   "price":3500, "stock":15, "rating":4.7},
]
products.insert_many(docs)
print("Inserted", products.count_documents({}), "documents\n")

# --- COUNT: how many Electronics products? ---
print("Electronics count:", products.count_documents({"category":"Electronics"}))
```

```
# --- SORT + LIMIT: the 3 most expensive products ---
# {"_id":0,...} hides the internal _id and shows only name and price
print("\nTop 3 most expensive:")
for d in products.find({}, {"_id":0,"name":1,"price":1}).sort("price", -1).limit(3):
    print("  ", d)

# --- SKIP + LIMIT: paging. Sort by price, skip the 3 cheapest, show next 3 ---
print("\nPage 2 (skip 3, limit 3) by cheapest first:")
for d in products.find({}, {"_id":0,"name":1,"price":1}).sort("price", 1).skip(3).limit(3):
    print("  ", d)
```

```
python3 ~/mongo_task_a.py
```

Task b - Aggregation pipeline for summary statistics

Save as ~/mongo_task_b.py. An aggregation pipeline is a Python list of stages; each stage is a dictionary.

```
#!/usr/bin/env python3
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
products = client["bdalab"]["products"]

# --- Pipeline 1: per-category summary, sorted by average price (high to low) ---
pipeline1 = [
    {"$group": {                        # group all documents by category
        "_id": "$category",
        "count":      {"$sum": 1},        # number of products
        "avgPrice":   {"$avg": "$price"}, # average price
        "totalStock": {"$sum": "$stock"}  # total stock
    }},
    {"$sort": {"avgPrice": -1}}          # order groups by avgPrice desc
]
print("Per-category summary:")
for d in products.aggregate(pipeline1):
    print("  category={_id:<12} count={count} avgPrice={avgPrice:.2f} "
          "totalStock={totalStock}".format(**{**d, '_id': d['_id']}))

# --- Pipeline 2: $match then $group. Average rating of Electronics only ---
pipeline2 = [
    {"$match": {"category": "Electronics"}},   # keep only Electronics
    {"$group": {
        "_id": "$category",
        "avgRating": {"$avg": "$rating"},
        "items":     {"$sum": 1}
    }}
]
print("\nElectronics rating summary:")
for d in products.aggregate(pipeline2):
    print("  ", {k:(round(v,2) if isinstance(v,float) else v) for k,v in d.items()})
```

```
python3 ~/mongo_task_b.py
```

Output:

Sample outputs (representative terminal screenshots).

Task a - count, sort, limit, skip

```
(bdaenv) student@ubuntu: ~

$ python3 ~/mongo_task_a.py
Inserted 10 documents


Electronics count: 4
```

```
Top 3 most expensive:
   {'name': 'Laptop', 'price': 75000}
   {'name': 'Monitor', 'price': 12000}
   {'name': 'Jacket', 'price': 3500}

Page 2 (skip 3, limit 3) by cheapest first:
   {'name': 'Mouse', 'price': 800}
   {'name': 'Rice Bag', 'price': 1200}
   {'name': 'Keyboard', 'price': 1500}
```

Task b - aggregation pipeline

```
(bdaenv) student@ubuntu: ~

$ python3 ~/mongo_task_b.py
Per-category summary:
   category=Electronics  count=4 avgPrice=22325.00 totalStock=277
   category=Clothing     count=3 avgPrice=2100.00 totalStock=120
   category=Grocery      count=3 avgPrice=503.33 totalStock=540

Electronics rating summary:
   {'avgRating': 4.38, 'items': 4, '_id': 'Electronics'}
```

The results show Electronics is the most expensive category on average (22,325) while Grocery holds the most stock (540 units), and Electronics products carry a high average rating of 4.38 - insights computed entirely inside MongoDB by the aggregation pipeline.

LAB#8

Title: "Data Preprocessing with Apache Spark"

Aim:

To load a structured student dataset into an Apache Spark DataFrame and perform data preprocessing so the data becomes clean and analysis-ready. The tasks are:

1. Load a CSV student dataset into a Spark DataFrame.
2. Handle missing values in the data.
3. Remove duplicate records.
4. Convert columns to their correct data types.
5. Apply basic transformations (create new derived columns).

Theory:

What is Apache Spark and a DataFrame?

Apache Spark is a fast, modern engine for processing large data. Where Hadoop MapReduce writes intermediate results to disk between steps, Spark keeps data in memory, which makes it far quicker for the repeated passes that data cleaning and analysis require. Spark can run the same code on one laptop or on a thousand-machine cluster.

Spark's main tool for structured data is the DataFrame - a table with named, typed columns, very similar to a spreadsheet or a Pandas DataFrame, but able to be split across many machines. We use PySpark, Spark's Python interface, so all the code stays in Python.

Why data preprocessing matters

Real datasets are almost never clean. They contain blank cells (missing values), repeated rows (duplicates), and numbers that were read as text. If we analyse such data directly, the results are wrong or the program crashes. Preprocessing is the essential first step of turning messy raw data into trustworthy data. Each cleaning task fixes one specific problem:

Type conversion: When a CSV is read, every column is text by default. Scores like '78' must be converted to real integers before we can average them; otherwise 'greater than' and 'average' make no sense.

Removing duplicates: The same record entered twice would be counted twice, distorting every total and average. We keep only one copy of each.

Handling missing values: A blank score cannot simply be ignored. A common, sensible fix is to fill each blank with the average of that column, so the record can still be used without skewing the overall picture.

Transformation: We often add useful derived columns, such as each student's total and average marks, computed from the existing columns.

Algorithm:

1. Start a SparkSession (the entry point to Spark) and read the CSV into a DataFrame.
2. Cast the numeric columns from text to integers.
3. Remove duplicate rows with dropDuplicates().

4. For each numeric column, compute its average and use it to fill the missing (null) values in that column.
5. Add two derived columns, 'total' and 'average', from the subject marks.
6. Show the cleaned DataFrame and save it for the next experiment.

Code:

Dependencies to install for this experiment

```
# Java is required by Spark (already installed in Experiment 1).
# Install PySpark and Pandas inside the virtual environment:
source ~/bdaenv/bin/activate
pip install pyspark pandas

# If Spark complains it cannot find a host address, set this once:
export SPARK_LOCAL_IP=127.0.0.1
```

Create the sample (messy) dataset

Save as ~/students.csv. It deliberately contains blanks (missing values) and one duplicate row (student 103 appears twice).

```
cat > ~/students.csv << 'EOF'
student_id,name,age,gender,math,science,english,attendance
101,Anil,17,M,78,82,,95
102,Bhavna,18,F,90,88,91,98
103,Chandra,,M,55,60,58,80
104,Deepa,17,F,,75,80,88
105,Esha,19,F,66,,70,72
106,Farhan,18,M,45,50,48,60
107,Gita,17,F,88,92,85,99
108,Hari,20,M,72,,68,,
103,Chandra,,M,55,60,58,80
109,Iqbal,18,M,95,98,99,100
110,Jaya,,F,60,65,,85
EOF
```

Preprocessing script

Save as ~/spark_preprocess.py.

```
#!/usr/bin/env python3
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import IntegerType

# 1) Start Spark. 'local[*]' means use all cores of this one machine.
spark = (SparkSession.builder
         .appName("StudentPreprocessing")
         .master("local[*]")
         .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")   # hide Spark's noisy INFO logs

# 2) Read the CSV. header=True uses the first row as column names.
df = spark.read.csv("/home/student/students.csv", header=True, inferSchema=False)
print("RAW: rows =", df.count(), "columns =", len(df.columns))
df.show()

# 3) TYPE CONVERSION: cast text columns to integers
int_cols = ["student_id", "age", "math", "science", "english", "attendance"]
for c in int_cols:
    df = df.withColumn(c, F.col(c).cast(IntegerType()))

# 4) REMOVE DUPLICATES: drop identical rows
```

```
before = df.count()
df = df.dropDuplicates()
print("Removed", before - df.count(), "duplicate row(s)")

# 5) HANDLE MISSING VALUES: fill each numeric column's blanks with its average
for c in ["age", "math", "science", "english", "attendance"]:
    mean_val = df.select(F.round(F.avg(c)).alias("m")).collect()[0]["m"]
    df = df.fillna({c: int(mean_val)})
    print(f"Filled missing {c} with average = {int(mean_val)}")

# 6) TRANSFORMATION: add 'total' and 'average' marks columns
df = df.withColumn("total", F.col("math") + F.col("science") + F.col("english"))
df = df.withColumn("average", F.round(F.col("total") / 3, 2))

print("\nCLEANED DATA:")
df.orderBy("student_id").show()

# 7) Save the cleaned data for Experiment 9 (one CSV file, with header)
df.orderBy("student_id").coalesce(1).write.mode("overwrite") \
  .option("header", True).csv("/home/student/students_clean")
print("Cleaned data saved to /home/student/students_clean/")

spark.stop()
```

```
python3 ~/spark_preprocess.py
```

Output:

Sample output (representative screenshot). Note the raw data shows 11 rows with several 'null' blanks; after cleaning there are 10 rows, no blanks, and two new columns.

```
(bdaenv) student@ubuntu: ~

$ python3 ~/spark_preprocess.py
RAW: rows = 11 columns = 8
Removed 1 duplicate row(s)
Filled missing age with average = 18
Filled missing math with average = 72
Filled missing science with average = 76
Filled missing english with average = 75
Filled missing attendance with average = 86

CLEANED DATA:
+----------+-------+---+------+----+-------+-------+----------+-----+-------+
|student_id|   name|age|gender|math|science|english|attendance|total|average|
+----------+-------+---+------+----+-------+-------+----------+-----+-------+
|       101|   Anil| 17|     M|  78|     82|     75|        95|  235|  78.33|
|       102| Bhavna| 18|     F|  90|     88|     91|        98|  269|  89.67|
|       103|Chandra| 18|     M|  55|     60|     58|        80|  173|  57.67|
|       104|  Deepa| 17|     F|  72|     75|     80|        88|  227|  75.67|
|       105|   Esha| 19|     F|  66|     76|     70|        72|  212|  70.67|
|       106| Farhan| 18|     M|  45|     50|     48|        60|  143|  47.67|
|       107|   Gita| 17|     F|  88|     92|     85|        99|  265|  88.33|
|       108|   Hari| 20|     M|  72|     76|     68|        86|  216|   72.0|
|       109|  Iqbal| 18|     M|  95|     98|     99|       100|  292|  97.33|
|       110|   Jaya| 18|     F|  60|     65|     75|        85|  200|  66.67|
+----------+-------+---+------+----+-------+-------+----------+-----+-------+

Cleaned data saved to /home/student/students_clean/
```

The duplicate row for Chandra was removed, every blank was filled with its column average (for example Anil's missing English became 75), and the derived 'total' and 'average' columns were added - leaving a clean, typed, analysis-ready DataFrame.

LAB#9

Title: “Structured Querying with Spark SQL”

Aim:

To register the cleaned Spark DataFrame from Experiment 8 as a temporary view and query it using Spark SQL and the DataFrame API. The tasks are:

1. Load the cleaned dataset and register it as a temporary SQL view.
2. Run Spark SQL queries for filtering, grouping and aggregation.
3. Perform the equivalent operations using the DataFrame API for comparison.

Theory:

What is Spark SQL?

Spark SQL lets us query a Spark DataFrame using ordinary SQL, exactly as we queried Hive in Experiment 4 - but running on Spark's fast in-memory engine. To use it, we register a DataFrame as a temporary view (giving it a name), and then we can write SELECT ... FROM view_name ... just like a database table. The view is 'temporary' because it exists only while the program runs; it is not stored permanently.

Spark offers two equivalent ways to ask the same question: writing SQL text, or chaining DataFrame API methods such as .filter(), .groupBy() and .agg(). Both produce the same answer and run through the same engine, so the choice is a matter of style. Showing both helps students see that SQL and the programmatic API are two faces of the same tool.

Purpose of the tasks

Registering a view is the bridge between preprocessing and analysis: it exposes the clean DataFrame to the SQL engine.

Filtering, grouping and aggregation are the core analytical operations - selecting the rows we care about, bucketing them into groups, and computing summaries such as counts and averages per group. These answer real questions like 'what is the average score by gender?' or 'how many students passed?'.

Algorithm:

1. Start a SparkSession and read the cleaned data saved by Experiment 8 (with the correct types inferred).
2. Register the DataFrame as a temporary view named 'students'.
3. Run three Spark SQL queries: average score and count per gender; the top 3 students by average; and a pass/fail count.
4. Reproduce one of these queries using the DataFrame API (groupBy + agg) to show the two styles are equivalent.
5. Print each result.

Code:

Dependencies to install for this experiment

```
# No new installs - this experiment reuses PySpark from Experiment 8.
source ~/bdaenv/bin/activate
export SPARK_LOCAL_IP=127.0.0.1     # only if Spark asked for it earlier
```

This experiment uses the folder `/home/student/students_clean` produced at the end of Experiment 8, so run Experiment 8 first.

Spark SQL script

Save as ~/spark_sql.py.

```
#!/usr/bin/env python3
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = (SparkSession.builder
         .appName("StudentSparkSQL")
         .master("local[*]")
         .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

# 1) Read the cleaned data from Experiment 8.
# inferSchema=True lets Spark detect the numeric types automatically.
df = spark.read.csv("/home/student/students_clean", header=True, inferSchema=True)

# 2) Register the DataFrame as a temporary SQL view called 'students'
df.createOrReplaceTempView("students")

# ---------- Spark SQL queries ----------
print("Q1: average score and student count per gender")
spark.sql("""
    SELECT gender, ROUND(AVG(average),2) AS avg_score, COUNT(*) AS n
    FROM students
    GROUP BY gender
    ORDER BY gender
""").show()

print("Q2: top 3 students by average")
spark.sql("""
    SELECT name, average FROM students
    ORDER BY average DESC
    LIMIT 3
""").show()

print("Q3: pass/fail count (pass if average >= 50)")
spark.sql("""
    SELECT CASE WHEN average >= 50 THEN 'PASS' ELSE 'FAIL' END AS result,
           COUNT(*) AS cnt
    FROM students
    GROUP BY CASE WHEN average >= 50 THEN 'PASS' ELSE 'FAIL' END
""").show()

# ---------- Same as Q1 but using the DataFrame API ----------
print("Q1 again, using the DataFrame API instead of SQL:")
(df.groupBy("gender")
   .agg(F.round(F.avg("average"),2).alias("avg_score"),
        F.count("*").alias("n"))
   .orderBy("gender")
   .show())

spark.stop()
```

```
python3 ~/spark_sql.py
```

Output:

Sample output (representative screenshot). The SQL query and the DataFrame API query give identical results for Q1, proving the two styles are equivalent.

```
(bdaenv) student@ubuntu: ~

$ python3 ~/spark_sql.py
Q1: average score and student count per gender
+------+---------+---+
|gender|avg_score|  n|
+------+---------+---+
|     F|     78.2|  5|
|     M|     70.6|  5|
+------+---------+---+


Q2: top 3 students by average
+------+-------+
|  name|average|
+------+-------+
| Iqbal|  97.33|
|Bhavna|  89.67|
|  Gita|  88.33|
+------+-------+


Q3: pass/fail count (pass if average >= 50)
+------+---+
|result|cnt|
+------+---+
|  FAIL|  1|
|  PASS|  9|
+------+---+


Q1 again, using the DataFrame API instead of SQL:
+------+---------+---+
|gender|avg_score|  n|
+------+---------+---+
|     F|     78.2|  5|
|     M|     70.6|  5|
+------+---------+---+
```

The analysis shows female students had a slightly higher average score (78.2) than male students (70.6), Iqbal was the top performer, and 9 of 10 students passed. Crucially, the SQL and DataFrame-API versions of Q1 returned exactly the same table, demonstrating that Spark SQL and the DataFrame API are two equivalent ways of expressing the same analysis.

**********END**********